

JAVA PRACTICAL

Library Management – OOP Concepts

■ Basic Information	Date: 24/07/2026
Language	Java
Topic	OOP Concepts

■ Objective / Aim

To develop a Java program demonstrating classes and objects, constructors, encapsulation, inheritance, method overloading, method overriding, abstraction and interfaces.

■■ Problem Statement

Create a Java program for a library environment. Define a Book class with constructors, getters, setters and displayBook(). Demonstrate inheritance using Person, Student and Faculty; method overloading using Area; method overriding using Vehicle, Car and Bike; abstraction using Shape, Circle and Rectangle; and an interface using Printable and Report.

■ Class and Interface Design

Book – stores book details. Person – parent class. Student and Faculty – inherited classes. Area – demonstrates method overloading. Vehicle, Car and Bike – demonstrate method overriding. Shape – abstract class. Circle and Rectangle – subclasses. Printable – interface. Report – implementation class.

■ Steps to do the task

1. Create the Book class.
2. Add constructors, getters, setters and displayBook().
3. Create Person, Student and Faculty for inheritance.
4. Create overloaded area() methods.
5. Create Vehicle, Car and Bike for overriding.
6. Create Shape, Circle and Rectangle for abstraction.
7. Create Printable and Report for interface demonstration.
8. Run the Library main class.

■ UML Class Diagram

```
Book
■■■ bookId : int
■■■ bookName : String
■■■ author : String
■■■ price : double
■■■ Book()
■■■ Book(int, String, String, double)
■■■ displayBook()
```

```
Person
■■■ Student
■■■ Faculty
```

```
Area
■■■ area(double)
■■■ area(double, double)
■■■ area(float, float)
```

```
Vehicle
■■■ Car
■■■ Bike
```

```
Shape <<abstract>>
■■■ Circle
■■■ Rectangle
```

```
Printable <<interface>>
■■■ Report
```

■ Source Code

```
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    Book() {}

    Book(int bookId, String bookName, String author, double price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }
    public String getBookName() { return bookName; }
    public void setBookName(String bookName) { this.bookName = bookName; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }

    void displayBook() {
        System.out.println("Book ID: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
    }
}

class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    int rollNo;

    Student(String name, int age, int rollNo) {
        super(name, age);
        this.rollNo = rollNo;
    }

    void displayStudent() {
        display();
        System.out.println("Roll No: " + rollNo);
    }
}

class Faculty extends Person {
    String subject;

    Faculty(String name, int age, String subject) {
        super(name, age);
        this.subject = subject;
    }

    void displayFaculty() {
        display();
        System.out.println("Subject: " + subject);
    }
}

class Area {
    double area(double r) {
        return Math.PI * r * r;
    }

    double area(double l, double b) {
```

```

        return l * b;
    }

    double area(float b, float h) {
        return 0.5 * b * h;
    }
}

class Vehicle {
    void display() {
        System.out.println("This is a vehicle");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a car");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a bike");
    }
}

abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double r;

    Circle(double r) {
        this.r = r;
    }

    double area() {
        return Math.PI * r * r;
    }
}

class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }

    double area() {
        return l * b;
    }
}

interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Printing Report");
    }
}

public class Library {
    public static void main(String[] args) {

        System.out.println("----- Book Class -----");
        Book b = new Book(101, "Java Programming",
                           "James Gosling", 450.0);
        b.displayBook();

        System.out.println("\n----- Inheritance -----");
        Student s = new Student("Lokesh", 20, 25);
        s.displayStudent();

        Faculty f = new Faculty("Ravi", 40, "Java");
        f.displayFaculty();

        System.out.println("\n----- Method Overloading -----");
    }
}

```

```

        Area a = new Area();
        System.out.println("Circle Area: " + a.area(5));
        System.out.println("Rectangle Area: " + a.area(10, 5));
        System.out.println("Triangle Area: " + a.area(10f, 5f));

        System.out.println("\n----- Method Overriding -----");
        Vehicle v = new Car();
        v.display();
        v = new Bike();
        v.display();

        System.out.println("\n----- Abstraction -----");
        Shape c = new Circle(5);
        Shape r = new Rectangle(10, 5);
        System.out.println("Circle Area: " + c.area());
        System.out.println("Rectangle Area: " + r.area());

        System.out.println("\n----- Interface -----");
        Printable p = new Report();
        p.print();
    }
}

```

■ Compilation & Execution Commands

```

javac Library.java
java Library

```

■■ Output Screen

```

----- Book Class -----
Book ID: 101
Book Name: Java Programming
Author: James Gosling
Price: 450.0

----- Inheritance -----
Name: Lokesh, Age: 20
Roll No: 25
Name: Ravi, Age: 40
Subject: Java

----- Method Overloading -----
Circle Area: 78.53981633974483
Rectangle Area: 50.0
Triangle Area: 25.0

----- Method Overriding -----
This is a car
This is a bike

----- Abstraction -----
Circle Area: 78.53981633974483
Rectangle Area: 50.0

----- Interface -----
Printing Report

```

■ Result

The program was compiled and executed successfully. The practical demonstrates classes, constructors, encapsulation, inheritance, method overloading, method overriding, abstraction and interfaces.